ALGORITHMS · UNDERGRADUATE COMPUTER SCIENCE

Greedy Algorithms and Recurrence Analysis

How local choices and recursive structure make hard problems tractable.

Made with GAMMA

TWO STRATEGIES

Two ways to make hard problems tractable

One commits early. The other makes the problem smaller.



Greedy algorithms

Commit to the best-looking next move, then prove that the choice can belong to an optimal solution.



Divide and conquer

Break a problem into smaller copies, solve them recursively, and combine their answers.

This unit connects shortest paths, spanning trees, sorting, multiplication, and recurrence solving.

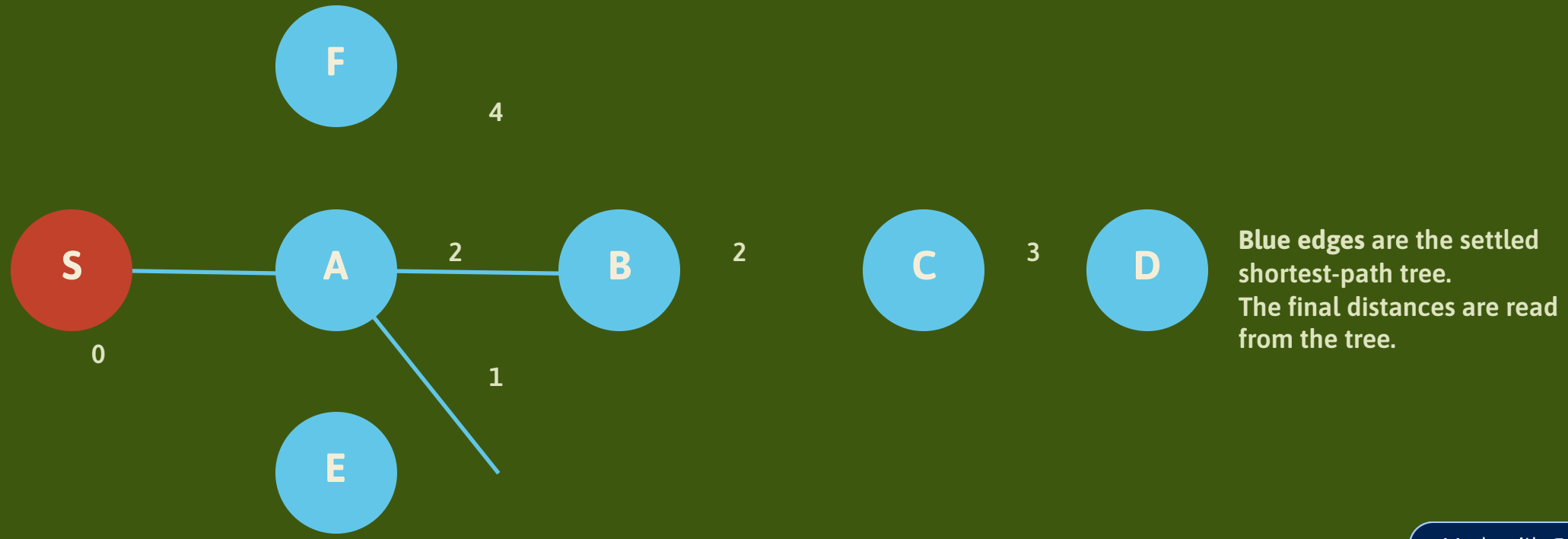
Made with GAMMA

GREEDY GRAPH ALGORITHM

Dijkstra builds a shortest-path tree

Initialize the source at distance 0. Then extract the unsettled vertex with the smallest tentative distance and relax its outgoing edges. Repeat until the destination or all reachable vertices are settled.

Required: nonnegative edge weights



Blue edges are the settled shortest-path tree.
The final distances are read from the tree.

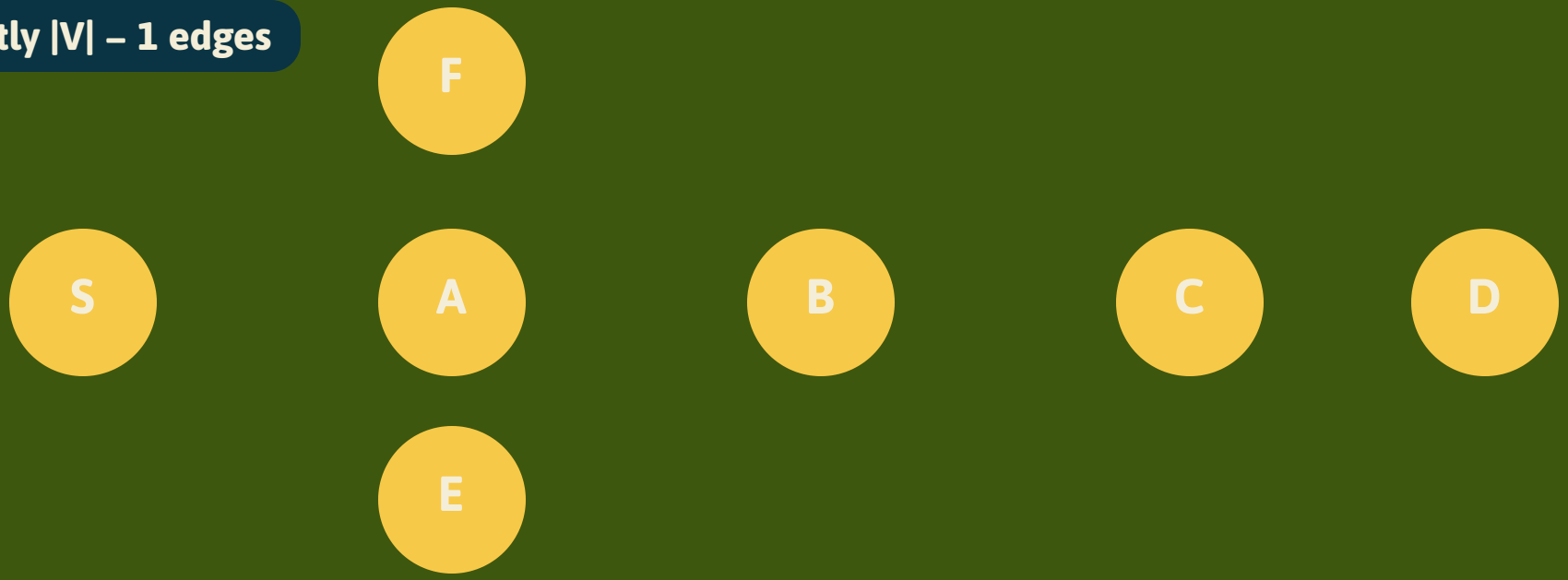
Made with GAMMA

GREEDY GRAPH ALGORITHM

A minimum spanning tree connects everything cheaply

The goal is different from shortest paths: connect every vertex with no cycles and minimum total edge weight.

Exactly |V| – 1 edges



Shortest path
Optimize one route from a source.

Spanning tree
Optimize the cost of the whole network.

Every chosen edge must preserve the possibility of an optimal completion.

Made with GAMMA

SAME OBJECTIVE · DIFFERENT GROWTH RULE

Prim and Kruskal grow the same kind of answer

Prim

Start from one vertex. Repeatedly add the cheapest edge leaving the current tree.

Trace: the boundary of one connected tree keeps moving outward.

Connected at every step

Kruskal

Sort edges by weight. Add the next lightest edge only when it does not create a cycle.

Trace: a union-find structure tracks which components are already connected.

Safe edge by cycle test

On the shared graph: both methods select a safe set of edges and finish with a minimum spanning tree. Their order of growth is the contrast.

Made with GAMMA

DIVIDE AND CONQUER

Merge Sort becomes a recurrence

The code structure becomes an equation.

Three moves

01

Split

Two halves of size n/2.

02

Sort

Two recursive calls.

03

Merge

Linear work to combine.

RUNNING TIME

$$T(n) = 2T(n/2) + \Theta(n)$$

The recurrence exposes the algorithm’s structure before it exposes the final bound.

Made with GAMMA

RECURRENCE ANALYSIS

The recursion tree turns repeated work into a picture

For Merge Sort, every level spends linear work.

1. Draw the root problem and its two subproblems
2. Label each level’s nonrecursive work
3. Count the depth until subproblems reach size 1
4. Add the work across levels

Root	One problem, size n; nonrecursive work $\Theta(n)$.
Level 1	Two problems, size n/2; combined work $\Theta(n)$.
Level 2	Four problems, size n/4; combined work $\Theta(n)$.
Continue	Each level contributes $\Theta(n)$ until size 1.

Depth: $\log n$ · **Total:** $\Theta(n \log n)$

Made with GAMMA

PATTERN MATCHING

The Master Method matches common recurrences

Start with the template, then check its assumptions.

TEMPLATE

$T(n) = aT(n/b) + f(n)$

Compare f(n) with $n^{\log_b a}$.

Case 1: Recursive work dominates. The leaves determine the growth.

Case 2: Contributions balance. Each level contributes at the same scale.

Case 3: Outside work dominates. Apply the regularity condition first.

Use the method as a pattern-matching tool, then verify that the recurrence fits its assumptions.

Made with GAMMA

A SECOND DIVIDE-AND-CONQUER EXAMPLE

Karatsuba reduces multiplication work

Split each number into high and low halves. Naive divide-and-conquer needs four half-size products.

Let x = ax + b and y = cy + d

$$xy = acB^2 + (ad + bc)B + bd$$

The cross-term appears to require two products: ad and bc.

Karatsuba computes

$$(a + b)(c + d) - ac - bd$$

One combined product plus the two outer products gives ad + bc.

One multiplication saved at every recursive level. The algebra changes the recurrence's growth from four subproblems to three.

Made with GAMMA

PROBABILISTIC ANALYSIS

Randomized Quick Sort trades certainty for a strong average

Quick Sort

1. Choose a pivot at random.
2. Partition around it.
3. Recurse on the smaller and larger partitions.

Consistently bad splits remain possible; balanced splits are common in expectation.

Expected Θ(n log n)

Quick Sort's average behavior

Merge Sort

1. Split deterministically.
2. Sort both halves.
3. Merge in linear work.

Its recurrence is deterministic: two half-size calls plus Θ(n) merge work.

Θ(n log n)

Merge Sort's worst-case guarantee

Compare Quick Sort's probabilistic recurrence with Merge Sort's deterministic one.

Made with GAMMA